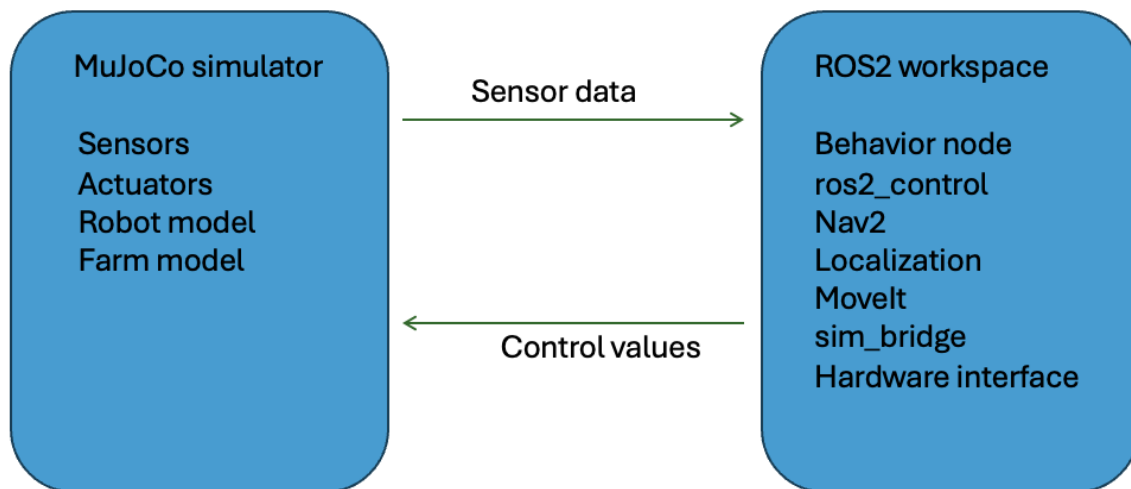


## system architecture

my project uses ROS2. in ROS2 there are multiple processes running parallelly. these processes are called nodes. they exchange data with each other through topics. once a node publishes a topic, other nodes can subscribe to this topic and access the published data. nodes can also offer services for quick operations (i.e. teleporting the food to shelves) or actions for prolonged operations (i.e. navigating to specified pose).

the system is based on abstraction of low levels operations such as sending control commands and reading sensors into higher level operation such as manipulating the arm and navigating. this abstraction could be seen from the [architectural diagram](#). in the package robot\_behavior there is a node that uses state machine to control the robot.

the simulation is based on the control loop: ROS2 receives sensor data from the simulation and in return the simulation receives from ROS2 control commands, as seen in the chart.

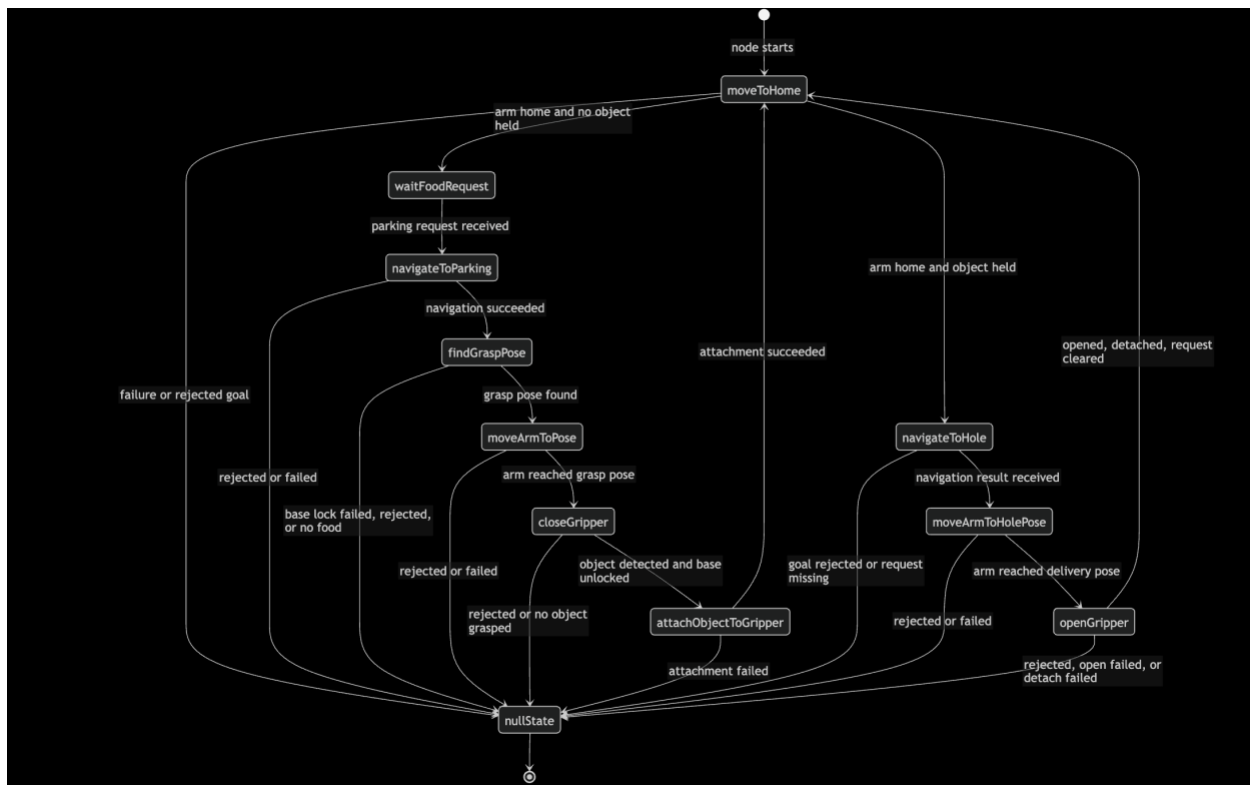


## main components and their interaction

the source code contains many packages written in python or C++. packages can launch nodes via launch files and/or provide their own code for nodes. they can also contain configurations and assets. package may contain code for multiple nodes or a single node. the following text is a layout of the main packages and the main nodes inside them.

the package robot\_behavior contains the node robot\_behavior. this node orchestrates all the high level components in order to control the robot. it uses the state machine in the

diagram:



the implementation separates the node into:

- behavior client responsible for providing convenient interface to send services and actions to other nodes.
- state machine class that initializes all the states with access to the behavior client.

the package `mujoco_ros2_control` is the bridge between the simulator and the controllers. the package implements the node controller\_manager responsible for loading controllers plugins and connecting them to the hardware interface. the package also provides the implementation of the hardware interface. it initializes and activates the simulation with all the sensors and export command and state interfaces. as part of the initialization, it creates a physics thread and the physics loop synchronizer that controls the speed of the simulation and adjust it to the system so that control commands won't be stale.

when all the controllers are active other parts of the system can send commands and receive information from the controllers (in ROS2, controllers are not just sending control commands but also collect sensor data such as IMU. this name is misleading). the controllers in my system are:

- joint state broadcaster. receives the joint states from the hardware interface and publishes them (the consumer of this topic is the robot state publisher which will be discussed in the robot\_description section).
- imu\_sensor broadcaster. receives the data from the imu sensors and publishes it as standard ROS2 msg.
- arm trajectory controller. offers an action to follow a specified arm trajectory. this trajectory will be specified by the node move\_group, which will be discussed later).
- gripper controller. enables to open and close the gripper.
- mecanum drive controller. receives velocity commands from topic published by the navigation stack and translates them to wheel commands and sends them to the hardware interface.

the package robot\_description contains the urdf of the robot with all the joints and the links. it also contains a launch description of a robot\_state\_publisher node. this node subscribes to the joint states topic from the joint state broadcaster and broadcasts transforms to all the links in the robot. other nodes can use this topic to know the state of the robot.

the package localization\_bringup contains configurations and assets for the localization nodes. it launches the map server, which publishes the map topic based on a pgm file. it launches the EKF node, which receives the imu and the odometry topics from the controllers and publishes a topic of filtered odometry. it launches an AMCL node with tuned configurations. the AMCL node estimates the position of the robot based of particle sampling. it broadcasts the transform map->base\_link to other nodes.

the package navigation\_bringup contains launch description of all of relevant nodes in the navigation stack. the main nodes it launches are:

- planner server that publishes a path to a desired target pose based on the dynamic global costmap. the costmap is constantly updated by configured plugins : the obstacle layer which subscribes to cameras and lidars and gets adds to the costmap the obstacles, static layer which represents fixed walls in the map, and inflation layer which expands the cost around obstacles.
- controller server that creates velocity commands to the mecanum drive controller. it uses the plugin MPPI, which samples many velocity commands and selects the best option based on configured criterions.
- collision monitor. filters the velocities before they get into the mecanum drive controller.

- `bt_navigator`. coordinates the navigation components and offers the action `navigate_to_pose`

the package `moveit_config` launches and configures the node `move_group`. using it directly is not convenient. the package `arm_behavior` uses the C++ library `MoveGroupInterface` to interact with this node. this package offers high level actions such as `MoveArmToPose` and `CheckPoseReachability`.

the package `gpd_ros2` contains a node that offers a service to find candidate grasp pose. it uses a known GPD library.

the package `grasp_pose_provider` offers an action to find grasp pose. it uses `open3d` to extract the food pointcloud, builds a service request for GPD, calls GPD, finds a reachable candidate grasp pose using `arm_behavior` and returns this pose. inside it, the class `GraspCandidateGenerator` is responsible for providing grasp candidates by using the GPD service, and the class `GraspCandidateValidator` is responsible for finding a reachable grasp pose out of all the candidate grasp poses using the action `CheckPoseReachability` offered by the `arm_behavior` node.

the package `simulation_interface_gui` provides a GUI for interacting with the simulation. it enables to user to throw the food on specified shelves in specified position and orientation and to send a request for the robot to carry food from some shelf. it also shows where the robot is in the map.

### **simulation scenarios**

[Scenario 1:](#) (click to see the recording)

an obstacle blocks the target shelf.

Expected outcome:

The robot will wait until the obstacle is gone and retry.

[Scenario 2:](#)

the robot grasps food and must deliver it to a feeding hole.

Expected outcome:

the robot will navigate to the hole pose and throw the food through the hole.

[Scenario 3:](#)

the scanned shelf is empty.

Expected outcome:

The robot aborts with status `FOOD_NOT_FOUND`

#### Scenario 4:

an object stands on the way to the shelf

Expected outcome:

the robot will move around the obstacle and reach the target pose.

#### performance metrics and KPI results

I measured the KPI criteria defined in the design review. the criteria were:

$$\text{collision rate} = \frac{\text{number of runs with collisions}}{\text{total number of runs}}$$

shelf scanning accuracy divides into 2 categories:

1. number of iterations in which a food was detected divided by the total number of iterations in which there was food.
2. number of iterations on which food wasn't detected out of the total number of iterations in which there wasn't food on the shelf.

grasp success rate: number of runs in which the robot managed to grasp food out of the total number of runs in which there was food on the shelf that the gripper could grasp.

delivery success rate: number of runs in which the robot managed to grab the food and throw it in the shelf out of the number of runs in which there was food on the shelf that the gripper could grasp.

the following table presents the result of the runs.

|                                | collision | grasp success | delivery success | accurate shelf scanning with food | accurate shelf scanning without food |
|--------------------------------|-----------|---------------|------------------|-----------------------------------|--------------------------------------|
| run 1                          | no        | yes           | no               | yes                               | -                                    |
| run 2                          | no        | no            | no               | yes                               | -                                    |
| run 3                          | no        | yes           | yes              | yes                               | -                                    |
| run 4                          | no        | yes           | no               | yes                               | -                                    |
| run 5                          | no        | no            | no               | yes                               | -                                    |
| run 6                          | no        | yes           | yes              | yes                               | -                                    |
| run 7                          | no        | yes           | no               | yes                               | -                                    |
| run 8                          | no        | yes           | yes              | yes                               | -                                    |
| run 9 (without food on shelf)  | no        | -             | -                | -                                 | yes                                  |
| run 10 (without food on shelf) | no        | -             | -                | -                                 | yes                                  |

|                                |    |   |   |   |     |
|--------------------------------|----|---|---|---|-----|
| run 11 (without food on shelf) | no | - | - | - | yes |
| run 12 (without food on shelf) | no | - | - | - | yes |

so the results are:

*collision rate* = 0% (no collisions)

*shelf scanning accuracy with food* = 100%

*shelf scanning accuracy without food* = 100%

*grasp success rate* = 75%

*delivery success rate* = 37.5%

### **challenges throughout the implementation**

I had a lot of challenges. ROS2 is inherently complex and mastering its interface is quite hard. much of the time was invested to understand it. Despite that ROS2 was designed for rebustness and scalability, as the system expanded many problems emerged.

I had problems in the simulation sychronization. due to limitation in GPU resources, the calculation of control commands was slow and the simulation had to be slower. otherwise the control commands would be stale. so I added pyhsics loop synchronizer.

one of the causes of slowing the simulation is that the arm trajectory planner doesn't always succeed. its algorithm is statistical and sometimes it succeeds and sometimes it doesn't. it took me a lot of time to figure it out. it is another reason for the simulation being slow. another cause of slowing the simulation is the GPD calculation, which takes about 4 minutes.

as a result of the simulation being slow, it took a lot of time to debug failiures.

due to the complexity of the system, each time something else triggered a failiure. I will list just a few of the causes:

1. initialy the forward direction of the robot was towards the positive y in the arena. but the convension in the mechanum drive controller is that the forward direction is the positive x. so there was a mismatch between the odometry topic published by the mechanum drive controller and the AMCL, which used the frame defined by the map assets.
2. I had mistakes in the colision matrix so there were many arm tolerance violations. it took a lot of time to understand which links that I added were wrong.
3. the controller server produced velocity commands that makes the robot collide with the wall. I had to tune the configurations and add colision monitor in order to prevent that.
4. the AMCL gave wrong pose. it took a lot of time to tune the configurations untill it gave

correct poses. it turns out that odometry is not very accurate so I had to increase the number of particles in the AMCL in order to get reliable localization.

each such problem took hours to debug and fix.